READING SHEET

# Middleware & Authentication

Module 3 · Frontend: React & Next.js · Day 42 · Week 9

| PROGRAM | WORK TYPE | EST. TIME |
|---|---|---|
| CodeOps · Full Stack | Revision + mini-project | 2–3 hours |

**Tip How to use this sheet**

Your endpoints have been anonymous until now. This sheet covers how they learn who is asking: middleware that redirects before a page renders, sessions held in a properly configured cookie, and three layers of protection — only one of which is actually security. Read Part 3 carefully; it is the part that keeps your users' data theirs.

## Part 1 · Middleware

One file at the project root, beside app/, that runs before every matching request reaches a page, layout or handler. It can rewrite the request, redirect it, or let it through — but it never renders anything itself.

middleware.js — project root

```
import { NextResponse } from "next/server";

export function middleware(request) {
  const session = request.cookies.get("session");
  if (!session) {
    const url = new URL("/signin", request.url);
    url.searchParams.set("next", request.nextUrl.pathname);
    return NextResponse.redirect(url);
  }
  return NextResponse.next();
}
```

A redirect here costs nothing: no page is built, no data is fetched, no layout runs. Compare the Day 31 guard, which had to mount a component before it could turn anyone away.

### The matcher

config.matcher — keep it narrow

```
export const config = {
  matcher: ["/checkout/:path*", "/orders/:path*"]
};

// or everything except static files:
// "/((?!_next/static|_next/image|favicon.ico).*)"
```

**Warning It runs on every match**

Without a matcher, middleware runs on every request in your application including images and static files — latency added to everything. Name the paths that actually need it, and keep the work inside tiny: middleware sits in front of the whole application, so a slow database call here slows every page.

### What it can and cannot do

| CAN | CANNOT |
|---|---|
| Read cookies and headers | Query a database directly |
| Redirect or rewrite a URL | Render a page or component |
| Set a cookie on the response | Use most Node.js APIs |
| Add or change request headers | Do heavy or slow work |
| Check geography or language | Be the only security you have |

Middleware runs on the **edge runtime** — close to the user and far from your database. The restrictions exist to keep it fast; they are not an oversight to work around. Verify cheaply here (does a session cookie exist, is it well-formed) and establish who the person is further in.

### Other uses

| USE | HOW | EXAMPLE |
|---|---|---|
| Redirect on auth | Read the cookie, redirect if missing | Signed-out visitor to /signin |
| Locale routing | Read Accept-Language, rewrite | Serving /am/menu to an Amharic browser |
| A/B tests | Set a cookie, rewrite to a variant | Two landing pages |
| Request context | Add a header the page reads | A request id for tracing |

A **redirect** changes the address bar and the person can see it happened. A **rewrite** keeps the URL and serves different content behind it, which is what makes locale routing and A/B testing invisible. A sign-in redirect should be visible; a variant should not.

## Part 2 · Sessions & Cookies

HTTP does not remember anyone. Authentication is the machinery that turns a stateless request into a known person, and almost all of it comes down to one cookie the browser sends back every time.

| # | WHAT HAPPENS WHEN SOMEONE SIGNS IN |
|---|---|
| 1 | The person posts their phone number and password |
| 2 | The server checks them against the stored hash |
| 3 | The server creates a session and sets it as a cookie |
| 4 | The browser sends that cookie with every later request |
| 5 | The server reads the cookie and knows who is asking |
| 6 | Signing out clears the cookie and invalidates the session |

> **Warning  Never store a password**
> Store a hash produced by bcrypt or argon2 — never the password itself, and never something you encrypted and could decrypt. Every authentication library you will ever use is doing exactly these six steps, which is why knowing the sequence lets you debug one when it behaves unexpectedly.

### The cookie flags that matter

setting a session cookie

```
cookies().set("session", token, {
  httpOnly: true,   // JavaScript cannot read it
  secure:   true,   // HTTPS only
  sameSite: "lax",  // not sent from other sites
  maxAge:   60 * 60 * 24 * 7
});
```

| FLAG | THE ATTACK IT BLOCKS |
|---|---|
| `httpOnly` | A script on your page stealing the session |
| `secure` | Reading it off an open network |
| `sameSite` | Another site acting as the user |

These are not optional. And do not put a token in `localStorage`: any script on the page can read it, and it is not sent automatically — a cookie with these three flags is both safer and less work.

### Two ways to hold a session

| DATABASE SESSION | SIGNED TOKEN (JWT) |
|---|---|
| Cookie holds an id | Cookie holds the claims |
| A lookup on every request | Verified without a lookup |
| Revoke instantly | Valid until it expires |
| Server holds the truth | The token holds the truth |
| Simple to reason about | Scales without shared state |

The trade is revocation. A signed token cannot be withdrawn before it expires, which is why token lifetimes are short and refresh tokens exist; a database session can be deleted the moment someone reports a stolen phone. Either is fine for this module — keep lifetimes short if you choose JWT, and keep the signing secret in an environment variable that is *not* prefixed NEXT_PUBLIC.

### Reading the session

lib/session.js — one helper, used everywhere

```
import { cookies } from "next/headers";
export async function getSession() {
  const token = cookies().get("session")?.value;
  return token ? verify(token) : null;
}
```

A page that renders, an action that writes and a handler that answers all read the session the same way — one helper means one place to change if the scheme does. Note that reading cookies opts

the route out of static rendering (Day 37), which is correct: a page that depends on who is asking cannot be prerendered.

## Part 3 · Protecting Routes & Data

Guarding a screen is not the same as guarding the data behind it. Three layers, each doing a different job — and only the innermost is security.

| LAYER | WHAT IT GIVES | IS IT SECURITY? |
|---|---|---|
| Middleware | A fast, cheap redirect | No — it only checks a cookie exists |
| Page or layout | The right screen for this person | Partly — it verifies the session |
| Action or handler | Refusal at the point of effect | Yes — this is the real check |
| Hiding the link | A tidy interface | No — not at all |

Middleware stops the pointless work of rendering a page nobody may see. The page check makes sure the screen matches the person. The check inside the action cannot be bypassed — and if you only had time to write one, that is the one.

### The page-level check

app/orders/page.js

```
export default async function OrdersPage() {
  const user = await getSession();
  if (!user) redirect("/signin?next=/orders");

  const orders = await getOrdersFor(user.id);
  return <OrderList orders={orders} />;
}
```

The important line is the query. Fetching by the **session's own id** makes it impossible to return someone else's orders, whatever the request contained. Putting the same check in a layout covers every route beneath it, which suits a whole private section.

### The check that actually matters

authentication, then authorisation

```
export async function cancelOrder(orderId) {   // "use server"
  const user = await getSession();
  if (!user) throw new Error("Not signed in");      // 401
  const order = await getOrder(orderId);
  if (order.userId !== user.id)
    throw new Error("Not yours");                   // 403
  await markCancelled(orderId);
}
```

> **Warning  Two different questions**
> Authentication asks who you are; authorisation asks whether you may touch this particular record. Passing the first tells you nothing about the second. Middleware only proved a cookie existed — every action and handler must re-establish who the person is and what they own.

### Returning them where they were

```
                                                          the "next" parameter, end to end
// middleware: remember the destination
url.searchParams.set("next", request.nextUrl.pathname);

// after a successful sign-in
const next = searchParams.next ?? "/";
if (!next.startsWith("/")) redirect("/");  // reject absolute URLs
redirect(next);
```

You built return-after-sign-in with router state on Day 31; here the destination lives in the URL, so it survives a refresh and works before any JavaScript runs. But never redirect to a value from the URL without checking it starts with a slash — otherwise anyone can craft a link that sends your users to another site after signing in. That is an **open redirect**, and the one-line check above is the whole fix.

## Part 4 · Authentication in Practice

You now understand the machinery well enough not to write it. A library handles the providers, the token rotation and the edge cases; your job is to place the checks and scope the queries.

| WHAT A LIBRARY HANDLES | WHY YOU WANT THAT |
|---|---|
| Providers and OAuth | Google or GitHub sign-in in an afternoon |
| Token signing and rotation | A subtle area, easy to get wrong |
| CSRF protection | A whole class of attack, handled |
| Session helpers | One call on server and client alike |
| Callbacks and roles | Somewhere to attach your own rules |

**Auth.js** (formerly NextAuth) is the standard choice for Next.js; Clerk, Lucia and Supabase Auth are alternatives, and all of them do the six steps from Part 2. Writing the flow yourself is an excellent exercise and a poor production decision — password reset, rate limiting and session fixation are all places to go wrong quietly.

### Roles

```
                                                          a role is a claim, not a component
{ id: "usr_31", name: "Almaz", role: "customer" }

// checked where the effect happens
if (user.role !== "staff") throw new Error("Not allowed");

// and reflected in the interface, separately
{user.role === "staff" && <Link href="/kitchen">Kitchen</Link>}
```

The same role answers two separate questions: what to show, and what to allow. Doing only the first gives you a tidy interface with no protection behind it. Two or three coarse roles is usually enough for a project this size.

### The mistakes to avoid

| MISTAKE | WHY IT FAILS |
|---|---|
| Only middleware protects it | It proved a cookie exists, nothing more |
| Trusting an id from the client | Anyone can send any id |

| MISTAKE | WHY IT FAILS |
|---|---|
| Checking auth in a client component | The person controls the browser |
| A token in localStorage | Readable by any script on the page |
| Redirecting to an unchecked URL | An open redirect out of your site |
| NEXT_PUBLIC on a secret | It is compiled into the bundle |

> **Tip  The habit to build**
> For every screen ask: what happens if someone calls this without the interface? If you cannot answer, the check is in the wrong place. Each of the six mistakes above is a five-minute audit of your own code — and they are the six things a reviewer will look for first.

## Assigned Reading

### Required

1. **nextjs.org — Authentication**  What / why: sessions, where to place checks, and the framework's own guidance on layering them.
2. **nextjs.org — Middleware**  What / why: the file, the matcher, the runtime limits, and redirect versus rewrite.

### Optional — go deeper

- **MDN — HTTP cookies**  — httpOnly, secure, sameSite and expiry, explained from the standard.
- **OWASP authentication cheat sheet**  — password storage, session handling and lockout, as a checklist.

## Exercises

Build the flow first, then try to break it.

1. Write a getSession helper that reads and verifies the session cookie.
2. Set the cookie on sign-in with httpOnly, secure, sameSite and a sensible maxAge.
3. Add middleware protecting /checkout and /orders, with a matcher covering only those.
4. Carry the intended destination in a next parameter and return to it after sign-in.
5. Reject any next value that does not start with a slash, then test a crafted link.
6. Scope the orders query to the session id and try to read another account.
7. Add a staff role and a /kitchen route, checked server-side rather than in the markup.

## Mini-Project · Securing Addis Eats

> **Tip  What you build**
> Sign-in and sign-out working end to end, middleware redirecting from the private routes, a single session helper used by pages, actions and handlers alike, ownership checked on every write, and a staff-only route. Then three attacks against your own application, documented with what happened.

### Requirements

- A session cookie with httpOnly, secure and sameSite set, and a getSession helper.
- Middleware with a narrow matcher, redirecting signed-out visitors and remembering the destination.
- The next parameter validated so an absolute URL cannot be used.
- Every private page verifying the session while rendering, not only in middleware.
- Queries scoped to the session id rather than an id from the request.
- Session and ownership both checked inside every action that writes.
- AUTH.md listing each protected route, its layers, and what each layer proves.

### The three attacks

- Call a server action from the console while signed out.
- Sign in as one account and call an action with another account’s order id.
- Craft a sign-in link with next pointing at an external site.

### Check yourself

- Can any script on your page read the session cookie?
- Does the orders page work if you change the id in the URL to someone else’s?
- Does middleware run on your image files? Check the matcher.
- Did all three attacks fail, and can you name the line that stopped each one?
- Is the staff link merely hidden, or actually refused server-side?
- Is your signing secret absent from the JavaScript the browser downloads?